

School of Advanced Technology
Final Year Project
Project Specification Report
项目说明报告 - 2026 年 7-8 月阶段

项目名称:	面向 MPM 分子标志物的 LLM 辅助知识图谱查询系统
学生姓名:	Liangyu Wei
学生 ID:	2360674
项目领域:	生物医学信息学 / 知识图谱 / AI 辅助信息系统

1- 项目描述与问题陈述

我选择这个项目，是因为恶性胸膜间皮瘤（MPM）的分子标志物信息比较分散，通常散落在不同论文和公共生物医学资源中。研究人员如果想判断某个 biomarker 是诊断、预后、治疗还是监测相关，还要继续查它和疾病、通路、药物、论文证据之间的关系，过程会比较零散。

我的项目目标不是做一个医学诊断模型，也不是让 AI 自由回答医学问题。我会把已整理的 MPM 标志物信息做成一个小型知识图谱查询原型，让系统用图数据库保存 biomarker-type-pathway-drug-paper-evidence 之间的关系，并让查询结果尽量可以追溯到证据句和论文。

第一阶段我会把重点放在一个可以运行、可以展示的原型上。整体路线是：收集少量种子数据 -> 用 CSV 整理和清洗 -> 设计知识图谱 schema -> 导入 Neo4j -> 编写 Cypher 查询模板 -> 做简单后端 API 和网页查询界面 -> 加入受控的 LLM 查询意图映射。

在系统中，Neo4j 负责存储图结构数据，Cypher 负责关系查询，Spring Boot 负责后端 API，前端页面负责输入和结果展示，LLM 只负责把用户自然语言问题转换成固定的查询意图和实体参数。最终结果仍然由 Neo4j 中的数据返回，而不是由 LLM 凭空生成。

原型架构：User Browser -> Vue/HTML Frontend -> Axios HTTP Request -> Spring Boot REST API -> Query Service / LLM Intent Service -> Neo4j Driver / Spring Data Neo4j -> Neo4j Knowledge Graph -> JSON result -> Table / Graph / Evidence display.

2- 项目目标与实现方法

总目标：我计划在 2026 年 7-8 月完成一个面向 MPM 分子标志物的知识图谱查询系统第一版原型。这个阶段的核心产出是 seed dataset、知识图谱 schema、Neo4j 原型数据库、Cypher 查询模板、简单查询界面，以及受控的 LLM 自然语言查询映射。

No.	Objective	Method / technical supplement
1	确认项目范围和 P13 系统需求。	我会把 7-8 月范围限定为第一版原型：seed dataset、schema、Neo4j、Cypher、简单界面和受控 LLM 查询映射。
2	整理 MPM 标志物种子数据。	我会整理小规模 seed dataset，包括 biomarker、type、pathway、drug、paper 和 evidence sentence。数据用 CSV 保存，并用 Python/pandas 做去重、标准化和空值检查。
3	设计知识图谱 schema。	我会定义 Biomarker、Disease、BiomarkerType、Pathway、Drug、Paper、Evidence 等节点，并用 HAS_TYPE、ASSOCIATED_WITH、INVOLVED_IN、TARGETED_BY、SUPPORTED_BY、HAS_EVIDENCE 表达关系。
4	实现 Neo4j 存储和 Cypher 查询。	我会把 CSV 导入 Neo4j，建立基本约束/索引，并写参数化 Cypher 模板，用于类型、证据、通路和药物/关系查询。用户输入不会直接拼接到查询字符串中。
5	构建后端 API 和简单查询界面。	我计划使用 Java Spring Boot 后端，采用 Controller、Service、Repository/DAO、DTO 分层。前端使用 Vue 3 或 HTML/CSS/JavaScript，通过 Axios 调用 API 并展示表格、证据句和关系图。
6	加入 LLM 辅助查询映射。	我会把 LLM 限制在 intent classification 和 entity extraction 上，只输出固定 JSON。后端校验 intent 和参数后，再调用白名单 Cypher 模板返回 Neo4j 结果。
7	完成测试记录和下一阶段计划。	我会用 Neo4j Browser、Postman 和测试问题检查数据、Cypher、API、LLM intent 和页面结果，并记录输入、预期输出、实际输出和改进点。

技术栈和框架补充

层级	我会使用的技术 / 框架	在本项目中的作用
数据处理	Python 3、pandas、CSV、Jupyter Notebook、正则表达式	整理论文证据和标志物数据；清洗 biomarker、paper、pathway、drug、evidence sentence 等字段；导出 Neo4j 可导入的 CSV。
知识图谱数据库	Neo4j Community / Neo4j Desktop、Cypher、Neo4j Browser	把 Biomarker、Disease、Type、Pathway、Drug、Paper、Evidence 建成节点，把语义关系建成边，并通过 Cypher 查询关系链。
后端框架	Java 17、Spring Boot、Spring Web、Spring Validation、Maven、Lombok	提供 REST API，处理用户查询、参数校验、业务逻辑、错误信息和结果封装。
Neo4j 连接	Neo4j Java Driver 或 Spring Data Neo4j	在后端执行参数化 Cypher，把 Neo4j 查询结果转换成 DTO / JSON 返回给前端。
前端界面	Vue 3、Vite、HTML/CSS/JavaScript、Axios，图谱展示可使用 ECharts 或 vis-network	实现搜索框、查询类型选择、结果表格、证据句展示和基础关系图展示。
LLM 辅助层	API 型 LLM、prompt template、JSON schema；Java 方案可了解 LangChain4j	只做自然语言问题到安全查询意图的转换，不直接生成医学结论。
测试与工程化	Postman、JUnit 5、Neo4j Browser、Git/GitHub、IntelliJ IDEA 或 VS Code；Docker 作为本地环境工具	检查 API、Cypher、数据导入、前端展示和 LLM intent；保留代码版本和环境配置。

知识图谱 schema 设计补充

我会把 Neo4j 当作关系网络数据库来设计，而不是把所有内容都放进传统表结构里。这样做的原因是 MPM biomarker 的重点不只是单个字段，而是 biomarker 与 disease、type、pathway、drug、paper 和 evidence 之间的连接关系。

节点 / 关系	字段或方向示例	说明
Biomarker	id, name, fullName, moleculeType	核心节点，例如 miR-126、mesothelin、fibulin-3。
Disease	id, name, abbreviation	疾病节点，例如 Malignant Pleural Mesothelioma / MPM。
BiomarkerType	diagnostic / prognostic / therapeutic / monitoring	标志物类型，用于分类查询。
Pathway / Drug / Paper / Evidence	name, target, title, doi, pmid, sentence	保存通路、药物、论文和具体证据句，方便结果追溯。
HAS_TYPE	(Biomarker)-[:HAS_TYPE]->(BiomarkerType)	表示某个 biomarker 属于哪种类型。
ASSOCIATED_WITH	(Biomarker)-[:ASSOCIATED_WITH]->(Disease)	表示 biomarker 与 MPM 相关。
INVOLVED_IN / TARGETED_BY	(Biomarker)-[:INVOLVED_IN]->(Pathway); (Biomarker)-[:TARGETED_BY]->(Drug)	表示通路、机制或药物/靶点关系。
HAS_EVIDENCE / SUPPORTED_BY	(Biomarker)-[:HAS_EVIDENCE]->(Evidence)-[:SUPPORTED_BY]->(Paper)	把查询结果和支持论文、证据句联系起来。

示例查询方向包括：查询 MPM 相关 diagnostic biomarkers；查询某个 biomarker 的 evidence sentence 和 supporting paper；查询某个 biomarker 参与的 pathway；查询某个 pathway 或 biomarker 关联的 drug/target。后端会使用参数化 Cypher，例如 \$disease、\$type、\$biomarkerName，而不是直接拼接用户输入。

LLM 辅助查询设计补充

我会把 LLM 设计成自然语言理解层。它的输出会被限制为 JSON，例如 {"intent":"search_by_type","entities":{"disease":"MPM","type":"diagnostic"}}。后端会检查 intent 是否在白名单中，也会检查 type、biomarker、pathway 等实体是否符合系统范围。如果问题超出范围，系统会返回无法处理该查询类型，而不是让 LLM 自由解释医学内容。

用户问题类型	LLM intent	后端调用
Which biomarkers are diagnostic for MPM?	search_by_type	按 Disease + BiomarkerType 查询 biomarker。
What papers support mesothelin?	search_evidence_by_biomarker	按 Biomarker 查询 Evidence 和 Paper。
Which pathway is miR-126 involved in?	search_pathway_by_biomarker	按 Biomarker 查询 Pathway。
Is fibulin-3 prognostic or diagnostic?	classify_biomarker_type	查询某 biomarker 的 HAS_TYPE 关系。

3- 项目计划：任务与里程碑（甘特图）

这个 specification 只覆盖今年 7-8 月阶段。我不会在这个阶段把项目扩展成大规模医学系统，而是先完成一个结构清楚、能运行、能演示的第一版原型。

Task / Milestone	7 月 W1-W2 2026	7 月 W3-W4 2026	8 月 W1-W2 2026	8 月 W3-W4 2026
确认项目范围和 P13 系统需求	X			
设计 CSV 字段和数据收集标准	X			
收集 MPM 标志物与论文种子数据	X	X		
清洗 CSV 数据并整理 evidence sentence		X	X	
设计知识图谱 schema 与示例图模型		X		
创建 Neo4j 数据库并导入节点/关系			X	
编写 Cypher 查询模板并在 Neo4j Browser 中测试			X	X
构建 Spring Boot 后端 API			X	X
构建简单前端查询页面和结果展示				X
完成 LLM intent JSON 映射原型				X
整理测试记录和下一阶段计划				X

4- 项目交付物

- 小规模 MPM 分子标志物 seed dataset，使用 CSV 格式整理，包含 biomarker、type、pathway、drug、paper 和 evidence sentence 等字段。
- 知识图谱 schema，覆盖 Biomarker、Disease、BiomarkerType、Pathway、Drug、Paper、Evidence 等节点，以及 HAS_TYPE、ASSOCIATED_WITH、INVOLVED_IN、SUPPORTED_BY 等关系。
- 一个已导入节点和关系的 Neo4j 原型数据库，并保留 Neo4j Browser 中的基本查询结果截图或测试记录。
- 几类可复用的 Cypher 查询模板，用于标志物查询、类型查询、通路查询、药物/靶点关系查询和证据检索。
- Spring Boot 后端 API 原型，能够接收前端请求、调用 Neo4j 查询并返回 JSON 结果。
- 简单网页查询界面，能够输入 biomarker 或自然语言问题，并展示表格结果、证据句和基础关系图。
- LLM 辅助查询映射原型，把自然语言问题转换成预定义 intent 和实体参数，再由后端调用安全的 Cypher 模板。
- 简短测试记录，包括 CSV 数据检查、Cypher 查询检查、Postman API 测试、LLM intent 测试样例和下一阶段改进计划。

5- 项目产业相关性

我认为这个项目和生物医学研究、制药信息管理以及科研知识管理都有关系。现实中的 biomarker 信息经常分散在不同论文、数据库和综述中，如果只靠人工阅读，很容易出现查找慢、关系不清楚、证据来源不容易追踪的问题。

知识图谱适合表达这类多跳关系，例如 biomarker 属于什么类型、和 MPM 是否相关、参与什么 pathway、是否连接到某些 drug/target，以及这些关系由哪些 paper 和 evidence sentence 支持。对研究人员来说，这类系统可以帮助他们更快整理候选标志物和证据链。

LLM 查询层的价值在于降低使用门槛。非技术用户不一定会写 Cypher 查询，但可以用自然语言提问。我的设计会让 LLM 只负责把问题转换成安全的查询意图，最终答案仍然来自整理后的 Neo4j 数据库。这比让 AI 直接生成医学结论更可控，也更符合一个原型系统在 7-8 月阶段应该完成的范围。

阶段边界：本阶段我不会训练深度学习医学诊断模型，也不会做大规模自动文献抽取或复杂 AI Agent。7-8 月的重点是把 Neo4j + Cypher + Spring Boot + 简单前端 + LLM intent mapping 的核心流程跑通。